

MiniChess Project Report

Student ID: 111061143

June 20, 2026

1 Overview

This project implements a MiniChess state evaluator, legal move generation, and a stronger benchmark policy. The main submitted files are `111061143_state.cpp`, `111061143_state.hpp`, `111061143_submission.cpp`, and `111061143_submission.hpp`. Additional integration files were also modified so the engine can register and use the stronger search policy through UBGI.

2 Modified Files

File	Purpose
<code>111061143_state.cpp</code>	Implements board evaluation and completes MiniChess legal move generation.
<code>111061143_state.hpp</code>	State interface used by the submitted implementation.
<code>111061143_submission.cpp</code>	Implements MiniMax, Alpha-Beta, Principal Variation Search, move ordering, quiescence search, and transposition table support.
<code>111061143_submission.hpp</code>	Declares the search policies and search parameters.
<code>111061143_registry.hpp</code>	Registers the available policies and changes the default policy to PVS.
<code>111061143_ubgi.cpp</code>	Connects the submitted registry to the UBGI engine and adds a small White opening book.
<code>111061143_ubgi.hpp</code>	Includes the submitted state header.
<code>111061143_cli.py</code>	Adds <code>alphabeta</code> and <code>pvs</code> to the command-line algorithm choices and changes the default White/Black algorithm from <code>minimax</code> to <code>pvs</code> .

3 State Implementation

The state implementation completes the missing MiniChess logic in the original starter code. In this project, a winning state means that a legal move can capture the opponent king. Once this happens, the game is already decided, so the evaluator returns the maximum score immediately instead of letting the opponent move the king away on a later turn. For non-terminal positions, the evaluator supports two modes:

- A simple material evaluator using piece values.
- A stronger evaluator using material, piece-square tables, king tropism, and mobility.

The stronger evaluator first locates both kings, then evaluates every piece from the current player's perspective. Material counts the value of pieces on the board. Piece-square tables give extra score for pieces on useful squares. King tropism gives a smaller positional bonus when pieces are closer to the opponent king; an immediate king capture is handled separately as a winning state. Mobility is calculated by generating legal moves for both sides and adding a bonus when the current player has more available moves.

The legal move generator was also completed for knights. The knight move table contains the eight standard L-shaped moves, such as two squares in one direction and one square sideways. For each candidate square, the code checks whether the square is inside the board and whether it is already occupied by a friendly piece. Legal empty-square moves and captures are added to the move list. If one of those captures takes the opponent king, the state is marked as a win and move generation returns immediately because the game is over.

4 Search Policy

The submitted policy keeps the original MiniMax interface but extends it with stronger search algorithms. The engine still plays only one move in the current turn, but it searches several future plies first. For example, depth 1 checks the engine's next move, depth 2 also checks the opponent reply, and depth 3 checks the engine's next response after that. This lookahead helps avoid moves that look good immediately but lose material or the king on the following turn.

4.1 MiniMax

The basic MiniMax implementation recursively evaluates child states to the requested depth. Scores are converted back into the current player's perspective after each child search. Terminal wins are scored as $P_MAX - ply$, which prefers faster wins. Draws and repeated positions use a draw contempt parameter, making the engine slightly prefer continuing play over accepting neutral repetition.

4.2 Alpha-Beta Search

Alpha-Beta pruning was added to reduce the number of explored nodes while preserving the same final minimax value. Alpha is the best score the current side has already found. Beta is the opponent's limit: if a branch becomes good enough for one player that the other player would avoid allowing it, the engine does not need to continue searching that branch. This pruning lets the engine search deeper than plain MiniMax in the same amount of time.

4.3 Principal Variation Search

Principal Variation Search (PVS) is used as the default algorithm. PVS is an optimized form of Alpha-Beta. It searches the first, best-looking move normally. For later moves, it first performs a cheaper narrow-window search that only asks whether the move can beat the current best score. If the answer is no, the move is skipped quickly. If the answer is yes, the move is searched again with a full window to get a more accurate score. This works well when move ordering puts a strong move first.

4.4 Move Ordering

Moves are sorted before search. The ordering gives priority to:

- The best move from the transposition table.
- Captures, using a captured-piece-value minus moving-piece-value heuristic.

- Pawn promotions.
- Moves toward the center.

Good move ordering improves both Alpha-Beta and PVS because stronger moves are searched earlier. Once a strong score is found, weaker branches can often be cut off sooner, which saves time and allows deeper search.

4.5 Transposition Table

A thread-local transposition table stores previously searched positions by state hash. This is useful because different move orders can sometimes reach the same board position. Each entry records how deeply the position was searched, the score, the type of score, and the best move found. An exact score means the stored value is the searched value. A lower bound means the real score is at least that good, and an upper bound means the real score is at most that good. Cached results are only reused when the old search depth is at least as deep as the current request, because a shallow result may miss future tactics. The stored best move is also tried first next time, improving move ordering.

4.6 Quiescence Search

At depth zero, the search does not stop immediately on all positions. If the position still has important tactical moves, the engine continues through noisy moves such as captures and promotions. This reduces the horizon effect. For example, if the engine stops while its queen can be captured immediately, it may evaluate the position too optimistically. Quiescence search looks at the immediate capture first, then evaluates a quieter and more stable position.

5 UBGI Integration

The registry adds three searchable policies: `minimax`, `alphabeta`, and `pvs`. The default algorithm is changed to `pvs`, so benchmark runs use the strongest submitted search by default.

The UBGI file also includes a small exact-position book for White. It starts from the opening position and may continue for many moves only if the board keeps matching the stored line. The book is keyed by encoded board state and side to move, so it is only used when the current position exactly matches a stored position. If the opponent makes a different move, the position no longer matches the book and the engine automatically falls back to normal PVS search.

6 Conclusion

The final engine improves the starter MiniChess implementation in two main ways. First, the game state is complete enough to evaluate positions and generate knight moves correctly. Second, the submitted benchmark policy replaces plain MiniMax with PVS, Alpha-Beta pruning, move ordering, transposition table caching, and quiescence search. These changes increase search strength while keeping the original project interfaces compatible with the provided benchmark and UBGI framework.

7 Notes

Codes are also upload on github